操作系统面试八股笔记

面向 Go 后端 / AI Agent 开发实习

操作系统面试八股笔记

一、硬件结构与 CPU 缓存

1. CPU 如何执行程序
2. 中断、异常与软中断
3. 存储层次与 CPU 缓存
4. 缓存一致性与伪共享
5. 局部性原理

二、操作系统结构

6. 内核态、用户态与系统调用
7. Linux 内核与 Windows 内核

三、内存管理

8. 虚拟内存要解决的问题
9. 分段
10. 分页与多级页表
11. TLB
12. 缺页中断与按需分页
13. 页面置换
14. Linux 虚拟地址空间
15. malloc、brk 与 mmap
16. free 是否把内存还给操作系统
17. 内存耗尽时会发生什么

四、进程、线程、调度与同步

18. 进程与状态
19. 线程
20. 上下文切换
21. 调度算法与 Linux 的调度器
22. 进程间通信
23. 互斥、信号量与条件变量
24. 自旋锁、读写锁、悲观锁与乐观锁
25. 死锁
26. 一个进程能创建多少线程
27. 线程崩溃，进程会怎样

五、文件系统

28. inode、dentry 与超级块

29. 虚拟文件系统与文件描述符
30. Page Cache 与「写完会不会丢」
31. 硬链接与软链接
32. 磁盘调度

六、设备管理

33. 设备与 I/O 控制方式
34. 从按下键盘到屏幕上的字符

七、网络系统

35. 阻塞、非阻塞、同步、异步
36. select、poll 与 epoll
37. Reactor 与 Proactor
38. 零拷贝

八、Linux 命令与性能排查

39. 先看整体，再拆资源
40. CPU
41. 内存
42. 磁盘
43. 网络
44. 一条可用的排查顺序

> 使用说明：按「主题 → 知识点」组织的结构化笔记。知识框架主要参考小林coding《图解系统》（硬件结构、操作系统结构、内存管理、进程与线程、调度、文件系统、设备管理、网络系统、Linux 命令），已重新整理改写。先记加粗关键词，再展开。Linux 行为随内核版本变化处已标明，面试时以当前内核和 `/proc` 实际值为准。

操作系统面试八股笔记

一、硬件结构与 CPU 缓存

1. CPU 如何执行程序

- 程序在磁盘上是静态文件。加载进内存后，CPU 按**程序计数器（PC）**取出指令，经译码、执行，再写回寄存器或内存。一条指令的生命周期通常是**取指 → 译码 → 执行 → 访存 → 写回**。
- **寄存器**速度最快、容量最小，用来放 PC、通用寄存器、标志寄存器和栈指针。编译器把热点变量尽量放在寄存器里。
- CPU 不直接理解 C/Go，源码要先编译成机器指令。函数调用会把返回地址和局部变量压栈，`return` 再弹栈，所以递归过深会栈溢出。
- 现代 CPU 用**流水线**重叠多条指令的各阶段，用**分支预测**和**乱序执行**填满流水线。预测失败要冲刷流水线，代价很高。

2. 中断、异常与软中断

- **中断**来自硬件（网卡收包、磁盘完成、键盘），用来打断当前指令流，让 CPU 去处理更紧急的事。
- **异常**来自 CPU 执行指令时（缺页、除零、非法指令）。缺页是异常，不是外设中断。
- **软中断（softirq）**是内核里的下半部机制：硬中断只做最少工作，把耗时处理推迟到软中断，避免长时间关中断。网卡收包的协议栈处理大量走软中断。
- `top` 里的 `hi` 是硬中断时间，`si` 是软中断时间。`si` 很高时，要看网络包是否过多（`/proc/softirqs`、`/proc/interrupts`）。

3. 存储层次与 CPU 缓存

- 速度从快到慢：**寄存器 → L1 → L2 → L3 → 内存 → SSD → HDD**。离 CPU 越近越快、越贵、越小。L1 通常分指令缓存和数据缓存，L3 多核共享。
- 缓存按**缓存行（cache line）**装数据，x86-64 上常见 **64 字节**。CPU 读一个字节，往往把周围 64 字节一起带上来。
- 命中则直接用缓存；未命中才访问下一级。写策略常见有写回（write-back，先改缓存，稍后刷内存）和写直达（write-through）。
- 磁盘比内存慢几个数量级：内存是电信号随机访问，机械盘还要寻道和旋转。所以操作系统用内存做 Page Cache，把「读磁盘」尽量变成「读内存」。

4. 缓存一致性与伪共享

- 多核各自有 L1/L2。同一地址若同时在多个核的缓存里，必须保证看到的值一致。常见协议是 **MESI**：Modified、Exclusive、Shared、Invalid。
- 一个核改了某缓存行，其他核上的副本要失效。这靠缓存一致性总线消息完成，会占用核间带宽。
- **伪共享（false sharing）**：两个核改的是不同变量，但它们落在同一条缓存行里，于是互相把对方的缓存行打失效，性能像在抢同一把锁。
- 避免办法：让高频写的变量按缓存行对齐、隔开（padding），或减少跨核写同一块热数据。读多写少时影响小。

5. 局部性原理

- **时间局部性**：刚用过的数据很可能马上再用。
- **空间局部性**：用过地址附近的数据很可能接着用。缓存行、Page Cache 预读、CPU 预取都建立在这一点上。
- 写得让 CPU 更快的代码：按行遍历二维数组（匹配内存连续布局）、缩小热点数据结构、避免在热循环里随机跳着访问大数组、减少伪共享和虚函数式的间接跳转。

延伸 / 易错点

- 缓存命中率不是「加了缓存就一定快」。随机访问大数组时，工作集超过缓存，命中率会掉下去。

二、操作系统结构

6. 内核态、用户态与系统调用

- CPU 有特权级。用户程序跑在**用户态**，不能直接操作硬件和任意物理内存；操作系统内核跑在**内核态**。
- 用户程序要读文件、发网络包、创建进程，必须通过**系统调用**陷入内核。过程是：用户态 → 保存现场 → 切到内核态执行内核函数 → 返回用户态。
- 切换本身有开销（寄存器、页表/TLB 相关状态、缓存污染）。所以「少做系统调用、把多次小调用合成一次」是常见优化，`sendfile`、批量 IO 都是这个思路。
- 库函数不等于系统调用。`malloc` 是 libc 函数，内部才可能调用 `brk`/`mmap`；`printf` 最终才走到 `write`。

7. Linux 内核与 Windows 内核

- **Linux** 是宏内核：进程调度、内存、文件系统、驱动大多在内核态同一地址空间。模块可以动态加载，但模块崩溃仍可能拖垮内核。优点是子系统间调用直接、路径短。
- **Windows**（NT）也是宏内核，把图形等放进内核以换性能；另有用户态子系统的历史分层。面试里常把「Linux 宏内核 vs 微内核」对比：**微内核**只把调度、IPC、最基础内存留在内核，驱动和文件系统在用户态，隔离更好，但 IPC 多、路径长（如 Minix、早期 Mach；seL4 是现代微内核）。
- Linux 用 `/proc`、`/sys` 把内核数据暴露成文件，这是虚拟文件系统，不在磁盘上。

三、内存管理

8. 虚拟内存要解决的问题

- 没有虚拟内存时，进程直接使用物理地址：多进程互相踩内存、程序加载地址被绑死、物理内存不够就跑不了大程序。
- **虚拟内存**让每个进程拥有独立的地址空间。CPU 发出的是虚拟地址，由 **MMU** 按页表翻译成物理地址。
- 带来的结果：进程之间隔离；程序员看到连续地址，物理上可以不连续；暂时不用的页可以换出，实现「用更少的物理内存跑更大的程序」。
- 同一段物理内存可以映射给多个进程（共享库、共享内存），也可以对一个进程映射成不同权限（代码页只读可执行，数据页可读写）。

9. 分段

- **分段**按逻辑单位划分：代码段、数据段、栈段，每段有基址和长度。地址是「段号 + 段内偏移」。
- 优点是符合程序结构，共享和保护可以按段做。缺点是段长不固定，容易产生**外部碎片**，换入换出也不方便。
- x86 历史上先有段，再在段内做分页。64 位长模式下分段基本被弱化，平坦地址空间加分页才是主流。

10. 分页与多级页表

- **分页**把虚拟地址和物理内存都切成固定大小的**页**，常见 **4KB**。地址分成「页号 + 页内偏移」。页内偏移不翻译，直接拼到物理页框上。
- 固定大小避免了外部碎片，但会产生**内部碎片**（一页没用满）。
- 单级页表太大：32 位、4KB 页时页表项就有约 100 万个。所以用**多级页表**：只为用到的虚拟地址分配下一级页表。64 位 Linux 常用 4 级（或 5 级）页表。
- **大页（huge page）** 2MB/1GB 能减少页表级数和 TLB 压力，适合数据库这类大块连续内存。

11. TLB

- 每次访存都查多级页表，等于多次访问内存。**TLB** 缓存最近的「虚拟页 → 物理页」映射。
- TLB 命中则不用走页表。进程切换会切换地址空间，旧 TLB 项必须失效（或使用带 PCID/ASID 的标记减少全刷）。
- 同一进程的线程共享地址空间，切换线程**不必切换页表**，这是线程切换比进程切换便宜的重要原因。

12. 缺页中断与按需分页

- 页表项可以标记「不在物理内存」。访问这样的页会触发**缺页异常**。
- 内核处理路径：找到这块虚拟页该对应的数据（零页、文件、swap）→ 分配物理页 → 填页表 → 返回用户态重新执行那条指令。
- **按需分页**：程序启动时不必把整个文件读入，第一次访问代码页或数据页才真正读盘。`malloc` 返回的地址也常常只是虚拟地址，第一次写入才分配物理页。
- 缺页分几种：软缺页（页在内存但页表未建好，或写时复制）、硬缺页（要读磁盘）、非法访问（地址不在 VMA 里，给进程发 SIGSEGV）。

13. 页面置换

物理页不够时，要选出牺牲页。面试要会的是**选择策略**，不要把教材算法说成 Linux 的实现。

算法	思路	特点
OPT	换出最久以后才用的页	理论最优，无法实现，用来做对照
FIFO	换出最早进入的页	简单，可能出现 Belady 异常
LRU	换出最久未使用的页	贴近局部性，精确实现要维护访问序，开销大
Clock / 二次机会	用访问位近似 LRU，指针转圈	硬件只提供访问位时很实用
LFU	换出使用次数最少的	受历史计数影响，不适应访问模式变化

- Linux 并不实现教科书上的严格 LRU。它用**活跃链表 / 非活跃链表**近似，并结合工作集、refault 距离判断预读进来的页是否值得留下。较新内核还有 **MGLRU**。说「Linux 用 LRU」在面试里要补上这句。
- **预读失效**：顺序读把后面的页提前读进来，若访问模式突然变了，这些页白白占内存。**缓存污染**：一次性扫过大文件，把真正的热点页挤出去。大文件传输因此常常要绕开 Page Cache（见零拷贝一节）。

14. Linux 虚拟地址空间

用户态从低地址到高地址大致是：

```
低地址
  代码段 text       机器指令，只读可执行
  数据段 data       已初始化的全局变量、静态变量
  BSS               未初始化的全局/静态变量，加载后清零
  堆 heap           malloc/brk 从低向高增长
  文件映射区        动态库、mmap 匿名映射、共享内存
  栈 stack          局部变量、调用帧，从高向低增长
高地址（用户空间顶）
  内核空间          每个进程映射同一份内核，仅内核态可访问
```

- 32 位 Linux 常见划分是用户 **3GB**、内核 **1GB**。64 位并不把 2^64 都用满，典型是用户和内核各 **128TB**，中间有未定义的空洞。
- 主线程栈大小常由 `ulimit -s` 决定，默认经常是 **8MB**。堆和 mmap 区才是动态伸缩的大头。
- 每个进程的内核地址映射的是同一块内核物理内存，所以陷入内核后不用再换一套内核页表。

15. malloc、brk 与 mmap

- `malloc` 不是系统调用，是分配器（glibc 默认 **ptmalloc**）在用户态管理内存池，需要更多虚拟地址时才调用 **`brk`/`sbrk`** 或 **`mmap`**。
- `brk` 移动堆顶，适合小块：分配器一次向内核要一大段，再切成小块复用。`mmap` 匿名私有映射在文件映射区单独划一块，适合大块，`free` 时可以 `munmap` 还回去。
- glibc 里有一个阈值（`M_MMAP_THRESHOLD`，经典默认 **128KB**）。小于阈值走堆，大于等于阈值走 `mmap`。阈值会被动态调整，不同 glibc 版本也不完全一样，不要背死「永远是 128KB」。
- `malloc` 得到的是**虚拟内存**。没被访问的页不会占用物理页，第一次写才缺页分配。这就是「4GB 物理内存的机器上 `malloc` 8GB 可以成功，但写满就会触发换页或 OOM」。

- 分配器在返回给用户的指针前面放 **chunk 头**（64 位上常见 16 字节，记录块大小）。`free(ptr)` 把指针向前退，读出大小，才知道该还哪一块。

16. free 是否把内存还给操作系统

- 经 `brk` 拿到的堆：`free` 通常只把块放回分配器的空闲链表，**不立刻缩小堆顶**。堆顶若被一块仍在使用的内存挡住，下面的空洞就还不回去。
- 经 `mmap` 拿到的大块：`free` 一般会 `munmap`，虚拟地址还给内核。
- 进程退出后，内核回收该进程的全部地址空间，不管用户有没有 `free`。
- 全用 `mmap`：小对象频繁系统调用、TLB 压力大。全用 `brk`：大块和碎片会把堆顶顶死，出现「有空闲却要不到连续空间」。所以两者组合使用。

17. 内存耗尽时会发生什么

- 匿名页可以换到 **swap**。swap 在磁盘上，能避免立刻杀进程，但访问换出页会非常慢（`vmstat` 的 `si`/`so`）。
- Linux 允许一定程度的**过量分配（overcommit）**。`malloc` 只记账，真正用完物理页加 swap 仍不够时，触发 **OOM Killer**，按启发式挑进程发 SIGKILL。
- 看是谁被杀：`dmesg` 里的 `Out of memory`。临时规避是加内存、限制进程、调 `oom_score_adj`，根因仍是泄漏或工作集过大。

延伸 / 易错点

- `free` 里的 `buff/cache` 大多可回收，判断内存紧不紧要看 `available`，不要只看 `free` 很小就认为内存光了。

四、进程、线程、调度与同步

18. 进程与状态

- **进程**是运行中的程序，是资源分配的单位：地址空间、打开的文件、信号处置。内核用 **PCB（Linux 里是 `task_struct`）** 描述它。进程消失，PCB 才消失。
- PCB 里有：pid、状态、优先级、内存描述符（页表）、打开文件表、寄存器现场、调度实体等。
- 基本状态：**创建 → 就绪 → 运行 → 阻塞（等待）→ 终止**。运行态占用 CPU；就绪态能跑但没排上；阻塞态在等 IO 或锁，给它 CPU 也跑不了。
- Linux 可运行任务在运行队列里；不可中断睡眠（D 状态）通常在等 IO，不响应信号，`ps` 里看到很多 D 往往是磁盘或网络文件系统卡住。

19. 线程

- **线程**是调度单位，一条执行流。同一进程的线程共享地址空间、文件描述符、堆；每个线程有自己的寄存器、程序计数器和栈。
- 创建、退出、切换都比进程轻：不用复制地址空间，不用切换页表。线程间传数据可以直接读写共享内存，不必进内核拷贝。
- 代价是隔离差：一个线程写坏堆、栈溢出踩到别的区域，整个进程一起死。
- Linux 实现是 **NPTL**：线程本质上也是 `clone` 出来的任务，共享 `mm_struct`，所以内核调度器看的是一个个任务，而不是「进程里的用户态线程」。Go 的 goroutine 是用户态协程，多个 goroutine 复用少量内核线程（GMP），和这里的内核线程不是同一层。

20. 上下文切换

- **进程切换**要保存/恢复寄存器，还要切换地址空间（页表）。页表一换，TLB 大面积失效，随后一段时间内地址翻译变慢。
- **同进程线程切换**保存/恢复寄存器和栈，不换页表，TLB 仍可用，所以更便宜。
- 切换本身不干活。线程太多时，切换和缓存失效会吃掉 CPU。自愿让出（锁、IO、`sleep`）和被迫让出（时间片用完、更高优先级醒来）都会切换。

21. 调度算法与 Linux 的调度器

先记通用算法，再记 Linux 实际用什么。

算法	要点
FCFS	先来先服务，护航效应：短任务被长任务堵住
SJF	短作业优先，平均等待时间好，但要预知长度，长任务可能饿死
时间片轮转	每个任务跑一个时间片，公平，时间片太小则切换过多
优先级	高优先级先跑，需要老化等办法防止饥饿
多级反馈队列	多个队列不同时间片，IO 型任务留在高优先级，CPU 型任务下沉

- Linux 长期使用 **CFS（完全公平调度）**：给每个任务一个虚拟运行时间 `vruntime`，每次选 `vruntime` 最小的跑。nice 值决定权重，权重高的 `vruntime` 涨得慢，于是实际分到的 CPU 更多。实现上长期用红黑树按 `vruntime` 排序。
- **Linux 6.6 起默认调度器改为 EEVDF**，仍追求公平，并更好地处理延迟敏感任务。面试说「Linux 用 CFS」时要补一句：6.6 之后默认是 EEVDF，思想仍是公平调度，不是时间片轮转的简单实现。
- 实时策略 `SCHED_FIFO` / `SCHED_RR` 优先级高于普通策略，用不好会饿死普通任务。

22. 进程间通信

方式	特点
匿名管道 pipe	半双工，只适合有亲缘关系的进程，数据是字节流，容量有限（常见默认 64KB，可调）
命名管道 FIFO	同管道，但有路径名，无亲缘关系也能用
消息队列	内核里按消息边界保存，有类型字段，容量受系统限制
共享内存	最快，数据不经内核拷贝；本身不同步，要另加信号量或锁
信号量	计数器，用来同步，不是用来传大量数据
信号 signal	异步通知，信息量很小；处理函数里只能调用异步信号安全的函数
套接字 socket	本机或跨机器，可双向，是最通用的 IPC，也是网络通信的接口

- 选法：本机大块数据用共享内存；简单父子进程用管道；要消息边界用消息队列或带长度头的 socket；跨主机只能 socket。

23. 互斥、信号量与条件变量

- **竞态**：多个线程同时读写共享数据，结果取决于调度顺序。临界区要用同步把操作变成原子的。
- **互斥锁 mutex**：同一时刻只允许一个线程进入临界区。谁没抢到就睡眠，交给调度器，不空转。
- **信号量**：计数。二元信号量可以当互斥锁；计数信号量用来限制同时进入资源的线程数（连接池、限流）。`P` 减一，减到负就等待；`V` 加一，叫醒等待者。
- **条件变量**：等待某个条件，而不是拿着锁空转。标准用法是：持锁检查条件 → 不满足则 `wait`（原子地放锁并睡眠）→ 被唤醒后重新检查条件（防虚假唤醒）→ 条件成立才继续。
- 唤醒用 `signal` 叫醒一个，`broadcast` 叫醒全部。改完条件再发信号，并且发信号时通常仍持有对应的锁，避免信号丢失。

24. 自旋锁、读写锁、悲观锁与乐观锁

- **自旋锁**：拿不到就忙等，不睡眠。临界区极短、不允许睡眠（中断上下文）时合适。临界区里做 IO 或睡眠会把 CPU 转死，也不能在单核上让持锁线程没机会跑。
- **读写锁**：多个读者可以同时进入，写者独占。读多写少时比互斥锁并发高。写多时退化为互斥，还有写者饥饿问题，需要公平策略。
- **悲观锁**：认为冲突很常见，先锁再操作。mutex、`SELECT ... FOR UPDATE` 都是。
- **乐观锁**：认为冲突少，先改，提交时检查版本或用 **CAS**。失败就重试。适合冲突低的计数器；冲突高时重试成本超过直接加锁。
- CAS 是 CPU 原子指令（比较并交换）。ABA 问题可以用版本号一起比较。

25. 死锁

四个必要条件同时成立才会死锁：

1. **互斥**：资源不能共享。
2. **持有并等待**：拿着 A 再去等 B。
3. **不可抢占**：别人不能把锁抢走。
4. **循环等待**：A 等 B、B 等 A。

- **预防**：破坏其中一个条件。最常用的是约定全局锁顺序，破坏循环等待；或者一次性申请全部资源，破坏持有并等待。
- **避免**：银行家算法在分配前判断会不会进入不安全状态。实现成本高，通用业务里少见。
- **检测与恢复**：记录资源分配图，发现环再回滚或杀进程。数据库死锁检测就是这条路。
- 写代码时：锁顺序一致、锁的粒度不要包住外部调用、给锁等待加超时、尽量不用嵌套锁。

26. 一个进程能创建多少线程

- 32 位上，默认 8MB 栈很容易把用户地址空间切完，几千个线程就到顶。
- 64 位地址空间很大，更早碰到的是：内核为每个线程准备的栈和 `task_struct`、pid 上限、`ulimit -u`、以及调度开销。
- 线程不是越多越快。CPU 密集型接近核数即可；IO 密集型可以更多，但到一定数量切换成本上升。Go 用 goroutine 就是为了避免「一个连接一个内核线程」。

27. 线程崩溃，进程会怎样

- 线程栈溢出、非法写内存、未捕获的信号（如 SIGSEGV），默认动作是**终止整个进程**，因为地址空间是共享的，继续跑其他线程也不安全。
- 只有线程自己 `return` 或 `pthread_exit`，才只结束该线程。
- 多线程里不该随意 `fork`：子进程只复制调用 `fork` 的那个线程，其他线程持有的锁可能再也不会释放。需要子进程时优先 `posix_spawn` 或在 `fork` 之后马上 `exec`。

五、文件系统

28. inode、dentry 与超级块

- 文件系统把磁盘分成**超级块、inode 区、数据块**。超级块记录文件系统类型、块大小、空闲块等，挂载时读入内存。
- **inode** 是文件的元数据：权限、属主、大小、时间戳、数据块位置。它不包含文件名。inode 号在一个文件系统内唯一标识文件。
- **目录**也是文件，数据块里存的是「文件名 → inode 号」列表。

- **dentry（目录项）**是内核在内存里的缓存，记录名字、inode 指针和父子关系，用来加速路径查找。dentry 不是磁盘上的目录文件。
- 多个 dentry 可以指向同一个 inode，这就是**硬链接**。

29. 虚拟文件系统与文件描述符

- **VFS** 在系统调用和具体文件系统（ext4、XFS、proc、NFS）之间提供统一接口：`open`/`read`/`write`/`close` 不关心底下是磁盘还是网络。
- `open` 返回的**文件描述符 fd** 是进程打开文件表的下标。内核还要维护系统级打开文件表（文件偏移、状态）和 inode。
- 多个 fd 或进程可以指向同一个打开文件描述，于是共享偏移；`dup`/`fork` 就是这种情况。各自 `open` 则偏移独立。
- 打开文件表项里有引用计数，最后一个 fd 关闭才真正释放。忘记 `close` 会耗尽 fd（`ulimit -n`）。

30. Page Cache 与「写完会不会丢」

- `read` 先查 **Page Cache**。没有再读磁盘，并留下缓存。内核还会**预读**后续连续页，所以顺序读远好于随机读。
- `write` 通常把数据拷进 Page Cache 就返回，页被标成脏页，由回写线程稍后写盘。所以 `write` 成功**不等于**已经落盘。
- 进程在回写前崩溃：只要机器还在，内核仍会把脏页写下去。**整机掉电**则 Page Cache 里的数据会丢。
- 需要持久化时调用 `fsync`（数据加必要元数据）或 `fdatasync`（偏数据）。数据库用 WAL 加 `fsync` 就是这个原因。
- ext4 默认 `data=ordered`：数据块先于元数据提交落盘，避免元数据指向垃圾块，但**不能代替 `fsync`**。
- `O_DIRECT` 绕过 Page Cache，适合自身已有缓存的数据库，也失去了内核预读和 IO 合并。

31. 硬链接与软链接

- **硬链接**：新目录项指向同一 inode，链接计数加一。不能跨文件系统，一般不允许给目录做硬链接。删掉一个名字，计数减一，到 0 且没有进程打开才释放数据。
- **软链接**：单独一个文件，内容是目标路径。可以跨文件系统，可以指向目录，目标消失后变成悬空链接。

32. 磁盘调度

- 机械盘的主要成本是寻道。调度算法用来重排请求：
  - **FCFS**：公平，磁头可能来回甩。
  - **SSTF**：先服务离磁头最近的，吞吐好，远处请求可能饿死。

- **SCAN / 电梯**：磁头沿一个方向扫到头再折返，避免饥饿。
- **C-SCAN**：只在一个方向服务，回头途中不服务，等待时间更均匀。

- SSD 没有寻道，这些算法的收益小很多，瓶颈更多在队列深度、GC 和并发。

六、设备管理

33. 设备与 I/O 控制方式

- **块设备**以块为单位随机访问（磁盘），走 Page Cache 和 IO 调度。**字符设备**以字节流访问（键盘、串口）。网络设备在 Linux 里单独处理，不完全套这两类。
- 控制方式，从 CPU 参与度由高到低：
  1. **轮询**：CPU 反复问设备好了没有，简单但浪费 CPU。
  2. **中断**：设备完成后再打断 CPU，适合不频繁的事件。
  3. **DMA**：设备直接和内存传数据，CPU 只下命令和处理完成中断。磁盘、网卡都走 DMA。
- 中断上下文里不能睡眠、不能做太长的工作，所以有上半部 / 下半部（软中断、tasklet、工作队列）。

34. 从按下键盘到屏幕上的字符

1. 键盘控制器产生中断，CPU 进入键盘中断处理，把扫描码放进缓冲区，尽快返回。
2. 内核把扫描码翻译成键码、字符，放进 tty 缓冲区，并唤醒正在 `read` 的进程。
3. 进程读到字符后，写回终端设备；显示路径再经驱动把像素/字符交给显卡。

- 这一路的重点是：**硬件中断只负责把事件收进来，复杂处理不在关中断的上半部里做。**

七、网络系统

35. 阻塞、非阻塞、同步、异步

这四个词描述的不是同一根轴。

- **阻塞 / 非阻塞**说的是调用方等不等：阻塞 `read` 在数据到来前不返回；非阻塞没有数据就立刻返回 `EAGAIN`。
- **同步 / 异步**说的是「谁来把数据搬好再通知你」：同步 IO 是调用期间由调用者等待完成；异步 IO（Windows IOCP、Linux `io_uring`）是提交请求后先返回，完成时由内核通知。
- 非阻塞加 `epoll` 仍然是**同步 IO**：就绪之后，用户程序自己再 `read`/`write`。不要把 IO 多路复用说成异步 IO。

36. select、poll 与 epoll

	select	poll	epoll
关心的 fd 怎么传	每次传入 `fd_set` 位图	每次传入 `pollfd` 数组	`epoll_ctl` 注册一次，保存在内核
数量上限	`FD_SETSIZE`，常见 1024	无固定 1024 上限	无固定上限，受内存和 `ulimit` 约束
就绪通知	每次线性扫全部 fd	同样线性扫	回调把就绪 fd 放进就绪链表，`epoll_wait` 只返回就绪的
适用	fd 很少	fd 较少	连接多、同时就绪的少（典型的服务器）

- **水平触发 LT**（epoll 默认）：只要缓冲区还有数据，每次 `epoll_wait` 都通知。编程简单。
- **边缘触发 ET**：只在状态变化时通知一次。必须把缓冲区读到 `EAGAIN`，并且 fd 要非阻塞，否则会丢事件。
- epoll 用红黑树保存被监视的 fd，就绪列表单独存放。它没有减少「数据从网卡到用户缓冲区」的拷贝，只减少了「每次调用都把全部 fd 拷进内核再扫一遍」的开销。
- fd 很少时，select/poll 未必更差。epoll 的优势在大规模连接。

37. Reactor 与 Proactor

- **Reactor（反应器）**：一个或多个线程用 `epoll` 等着事件，事件到了再分发给处理函数，由应用自己 `read`/`write`。Redis、Netty、Nginx 的事件模型都是 Reactor。可以是单线程 Reactor，也可以是「主 Reactor 接受连接 + 从 Reactor 处理 IO」。
- **Proactor（前摄器）**：应用提交读/写请求，内核做完数据搬运，再把「完成事件」交回来。Windows IOCP 是典型实现。Linux 的 `io_uring` 也是完成事件模型，比老的 `aio` 实用。
- 区别一句话：Reactor 通知的是「可以读了」，Proactor 通知的是「已经读好了」。

38. 零拷贝

传统 `read` + `send` 传文件：4 次上下文切换（两次系统调用），4 次拷贝（DMA 磁盘→内核、CPU 内核→用户、CPU 用户→socket 缓冲、DMA socket→网卡）。用户态并不修改文件内容时，中间两次 CPU 拷贝是多余的。

- **`mmap` + `write`**：内核缓冲区映射到用户空间，少一次 CPU 拷贝，仍是两次系统调用，内核到 socket 缓冲区仍要 CPU 拷一次。
- **`sendfile`**（Linux 2.1+）：一次系统调用，内核里把页缓存拷到 socket 缓冲区，上下文切换降到 2 次，拷贝仍有 1 次 CPU + 2 次 DMA。
- 网卡支持 **SG-DMA** 时（Linux 2.4+），`sendfile` 只把页描述符交给网卡，由 DMA 直接从页缓存发到网卡。内存里不再做 CPU 拷贝。这才是通常说的零拷贝：**CPU 不搬运数据**，不是一次拷贝都没有。

- Kafka 的 `FileChannel.transferTo`、Nginx 的 `sendfile on` 用的就是这条路径。
- 零拷贝依赖 Page Cache，适合小文件和会重复读的文件。传 GB 级大文件会把 Page Cache 占满，挤掉热点小文件，这时应改用**异步 IO + 直接 IO**，绕开页缓存。Nginx 可以用 `directio` 按文件大小切换。
- `sendfile` 不能在用户态修改内容。要压缩、加密再发送，就得走用户缓冲区，或者在内核里做（如 `splice` 配合管道，仍有限制）。

八、Linux 命令与性能排查

39. 先看整体，再拆资源

- `uptime` 的**负载**是 1/5/15 分钟内「正在跑 + 不可中断睡眠」的任务平均数，不是 CPU 利用率。核数是 4 时，负载长期大于 4 才说明排队。
- 负载高但 CPU 利用率低：多半有任务堵在 IO（D 状态），而不是算力不够。
- `vmstat 1` 把 CPU、内存换页、块设备 IO 放在一行里，适合先扫一眼：`us` 用户、`sy` 内核、`id` 空闲、`wa` 等 IO、`si/so` 换页、`bi/bo` 块设备吞吐。

40. CPU

- `top` / `pidstat -u 1`：看是哪个进程、用户态还是内核态。`sy` 高要怀疑系统调用过多、锁或软中断。
- `perf top`：看热点函数。再决定要不要 `perf record` 看调用栈。
- 软中断高：`cat /proc/softirqs`，网络场景很常见。

41. 内存

- `free -h` 看 `available`。`pidstat -r 1` 看谁的 RSS 在涨。
- `vmstat` 的 `si/so` 持续非 0 说明在换页，延迟会很差。
- 怀疑泄漏：对比进程 RSS 和业务上的对象数量，再用分配器或 `pprof`（Go）看分配栈。进程退出后 RSS 会全部还给内核，这点要和「free 不还堆顶」区分开。

42. 磁盘

- `iostat -x 1`：`%util` 接近 100% 且 `await` 很高，磁盘忙。机械盘和 SSD 的 await 量级差很多，要跟基线比。
- `iotop` 看是谁在写。写很多但业务认为已经 `fsync` 的，核对是不是只 `write` 进了 Page Cache。

43. 网络

- `ss -s` 看连接状态汇总，`ss -lnt` 看监听，`ss -antp` 看具体连接。

- LISTEN 行的 `Recv-Q` 是当前全连接队列长度，`Send-Q` 是队列上限。非 LISTEN 行的 `Recv-Q/Send-Q` 才是接收缓冲区未读字节和已发送未确认字节。
- `netstat -s` 里的 `listen queue overflow`、`SYNs to LISTEN sockets dropped` 用来判断全连接队列和半连接是否在丢。
- `sar -n DEV 1` 看网卡吞吐和错误包。

44. 一条可用的排查顺序

1. `uptime`、`vmstat` 判断是 CPU、IO 还是内存换页。
2. 用 `pidstat` / `top` 定位进程。
3. CPU 用 `perf`，IO 用 `iostat`，网络用 `ss` 和 `netstat -s`，进程行为用 `strace -c` 看系统调用分布。
4. 对照 `/proc/<pid>/limits`、`fd` 数量和 dmesg，确认不是 fd、内存或 OOM 先到了上限。

指标是累计值的（`netstat -s` 的丢包计数）要隔几秒看**增量**，不要看绝对值。